

Reviewer Pack — Algebraic Independence of Polynomial Systems over Fields and...

Entry a28e0714bbd5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 23:17:00 UTC

Algebraic Independence of Polynomial Systems over Fields and Boolean Circuit Monotonicity

Entry ID: a28e0714bbd5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 23:17:00 UTC

1. Conjecture

Field A (mathematical branch): Model Theory (Algebraic Independence) **Field B** (complexity object): Complexity Theory: Boolean Circuit Monotonicity

Statement:

{‘sentence_1’: ‘For every polynomial system F over a field K with at least two elements, the minimum degree of an irreducible polynomial that is algebraically independent over K is at most the maximum monotone degree of any circuit computing the permanent of F .’, ‘sentence_2’: ‘Equivalently, for a given polynomial system F and a constant c , if there exists a circuit C with monotone degree c that computes the permanent of F , then F has an irreducible polynomial in K that is algebraically independent over K and has degree at most c .’, ‘sentence_3’: ‘This does not hold for all fields K or all polynomial systems F , as demonstrated by counterexamples where no such irreducible polynomial exists.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Algebraic independence is a well-studied concept in model theory that can provide insights into the complexity of computational problems.’, ‘sentence_2’: ‘The monotone degree of a circuit has been used as a measure for its complexity in monotone circuits, and bounding this degree can potentially lead to improved algorithms or lower bounds.’, ‘sentence_3’: ‘This conjecture proposes a connection between these two areas that could reveal new structures or relationships between them.’}

Taxonomy category: MODEL_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5ac1dc42f3cde587

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all polynomial systems F over fields K with at least two elements and for each circuit C computing the permanent of F , the minimum degree of an irreducible polynomial that is algebraically independent over K is less than or equal to

the monotone degree of C . The conjecture is falsified if there exists a system F where this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'algebraic independence polynomial system field' AND 'Boolean circuit monotonicity' - 'minimum degree irreducible polynomial' IN ('field K') AND 'monotone degree circuit permanent' - 'circuit computing permanent' AND 'irreducible polynomial algebraically independent'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gcd(a, b):
        while b:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a*b) // gcd(a, b)

    def extended_gcd(a, b):
        if a == 0:
            return b, 0, 1
        else:
            g, y, x = extended_gcd(b % a, a)
            return g, x - (b // a) * y, y

    def mod_inverse(a, m):
        g, x, _ = extended_gcd(a, m)
        if g != 1:
            raise ValueError("Inverse doesn't exist")
```

```

    else:
        return x % m

def matrix_mul(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    if cols_A != rows_B:
        raise ValueError("Incompatible dimensions for matrix multiplication")

    C = [[sum(A[i][k] * B[k][j] for k in range(cols_A)) for j in range(cols_B)] for i in range(rows_A)]
    return C

def gaussian_elimination(matrix):
    n = len(matrix)
    m = len(matrix[0])
    augmented_matrix = [row + [1 if i == j else 0 for j in range(m, 2*m)] for i, row in enumerate(matrix)]

    for i in range(n):
        max_row = i
        for k in range(i+1, n):
            if abs(augmented_matrix[k][i]) > abs(augmented_matrix[max_row][i]):
                max_row = k

        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        factor = augmented_matrix[i][i]
        for j in range(m):
            augmented_matrix[i][j] /= factor

        for k in range(n):
            if k != i:
                factor = augmented_matrix[k][i]
                for j in range(m):
                    augmented_matrix[k][j] -= factor * augmented_matrix[i][j]

    return [row[m:] for row in augmented_matrix]

def is_irreducible(poly, field_size):
    degree = len(poly) - 1
    if degree == 0:
        return False

    for i in range(1, degree + 1):
        coeff = poly[i] % field_size
        if coeff != 0:
            return True
    return False

def min_irreducible_degree(poly, field_size):
    degree = len(poly) - 1
    for d in range(1, degree + 1):
        for i in range(degree - d + 1):
            sub_poly = poly[i:i+d]

```

```

        if is_irreducible(sub_poly, field_size):
            return d
    return degree

def permanent(matrix):
    n = len(matrix)
    if n == 0:
        return 1
    elif n == 1:
        return matrix[0][0]

    det = 0
    for j in range(n):
        sub_matrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        sign = (-1) ** (j % 2)
        det += sign * matrix[0][j] * permanent(sub_matrix)

    return det

def monotone_degree(circuit):
    max_depth = 0
    stack = []
    for gate in circuit:
        if gate == 'AND' or gate == 'OR':
            stack.append(gate)
            max_depth += 1
        elif gate == 'NOT':
            pass
        else:
            while stack and (stack[-1] == 'AND' or stack[-1] == 'OR'):
                stack.pop()
            if stack:
                max_depth -= 1
    return max_depth

def generate_polynomial_system(n, field_size):
    poly = [random.randint(0, field_size - 1) for _ in range(n + 1)]
    while not is_irreducible(poly, field_size):
        poly = [random.randint(0, field_size - 1) for _ in range(n + 1)]
    return poly

def generate_circuit(permanent_matrix):
    n = len(permanent_matrix)
    circuit = []
    for i in range(n):
        for j in range(n):
            if permanent_matrix[i][j] != 0:
                circuit.append('AND')
                circuit.extend([f'x{i}', f'x{j}'])
    return circuit

field_size = random.randint(2, 10)
n = random.choice([5, 10, 15, 20, 30, 40])

poly = generate_polynomial_system(n, field_size)
permanent_matrix = [[permanent([poly[j], poly[i]]) for i in range(n)] for j in range(n)]

```

```

circuit = generate_circuit(permanent_matrix)

irreducible_degree = min_irreducible_degree(poly, field_size)
monotone_deg = monotone_degree(circuit)

metric_name = "Minimum Irreducible Degree"
metric_value = irreducible_degree
instances_tested = 1
conjecture_holds = irreducible_degree <= monotone_deg
counterexample = "" if conjecture_holds else f"Counterexample: poly={poly}, circuit={circuit}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
        support_fraction = 1.0
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
        support_fraction = (len([r for r in results if r["conjecture_holds"]]) / len(results)) * 100

    print(f"RESULT: {'SUPPORTED' if all(r['conjecture_holds'] for r in results) else 'FALSIFIED'}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}

```

```

TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'Minimum Irreducible Degree', 'metric_value': 2, 'instances_tested': 1, 'conje
RESULT: SUPPORTED mean=2.0 std=0.0 support_fraction=1.00%

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only been conducted on a very small sample size ($n \leq 15$), which is insufficient to draw a general conclusion about the conjecture. The metric does not scale trivially with n , but without testing larger instances, we cannot confirm its validity.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: The test results indicate that for the instance tested, the conjecture holds true with a mean value of 2.0 and standard deviation of 0.0, meeting the | next: Further testing on larger polynomial systems is necessary to confirm the validity of the conjecture across a broader range of cases.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13047
2	preregistration	ollama_remote	glm4:latest	0	0	5819
3	novelty	ollama_remote	glm4:latest	0	0	4916
4	novelty	ollama_remote	glm4:latest	0	0	6910
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41052
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10647
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11362
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19058
9	critic	ollama_remote	glm4:latest	0	0	10442
10	judge	ollama_remote	glm4:latest	0	0	5629

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128882 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/a28e0714bbd5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/a28e0714bbd5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a28e0714bbd5.tar.gz` (if generated)